Lecture 02: Ports, Query Strings, and Node.js Modules

Request Routing and Shared Servers

Website Identification on Shared Servers

- When multiple websites are hosted on a single server, they share the exact same IP address.
- If these websites all use HTTPS, they also share the exact same default port.
- To determine which specific website the incoming request is meant for, the server reads the domain name explicitly attached to the request data (via the HTTP Host header and SNI protocols).

Proxy Routing and IP Masking

- For security, the actual backend server IP address is never directly exposed to the public.
- Instead, a substitute IP (acting as a reverse proxy) is provided to receive the traffic. This proxy inspects the requested domain name and silently routes the data to the correct internal application running behind the scenes.

Local Port Conflicts and Localhost

- **Local Port Conflicts:** When configuring a server to listen on a specific port (e.g., port 7000), an error may occur if another application is already actively listening on that exact port. To resolve this conflict, the backend code must be updated to use an available, unused port (e.g., changing to port 9000).
- **Localhost and its IP Address:** Just like a standard website name, localhost has an IP address. When accessing http://localhost:9000, the system resolves localhost to 127.0.0.1, its private loopback address. This is strictly a private, local IP used for internal testing on the same machine, and it is not accessible publicly over the internet.

Manual URL Routing and Input Validation

Values can be extracted directly from a URL path, but the input must be strictly validated to prevent server errors.

- **Simple Extraction:** A URL like /20 can be processed using request.url.slice(1) (or trim) to remove the leading slash and retrieve the number.
- **Handling Bad Input:** Users can input invalid data, such as text (/ab20), negative numbers, or numbers exceeding database limits (e.g., requesting a number greater than 100 when only 100 users exist).
- **Error Prevention:** If an error condition is met (like isNaN(number) or number <= 0 or number > 100), the logic must handle it and prevent subsequent lines of code from executing. This is managed using if, else if, and else blocks, or by using a return statement to exit immediately.

Building a Basic Calculator Route: A URL structured as /operation/firstNumber/secondNumber (e.g., /add/10/20) can be processed by splitting the string.

- Calling request.url.split('/') creates an array: ["", "add", "10", "20"].
- Specific elements are assigned to variables and processed using conditional statements.

- **Edge Cases:** Manual splitting requires checking if the path length is valid (e.g., avoiding errors with /div/4) and ensuring the extracted values are not NaN.

```
const server = http.createServer((request, response) => {

    const path = request.url.split('/');
    const operation = path[1];
    const number1 = Number(path[2]);
    const number2 = Number(path[3]);

    if (operation == 'add') {
        response.end(JSON.stringify(number1 + number2));
    } else if (operation == 'sub') {
        response.end(JSON.stringify(number1 - number2));
    } else if (operation == 'mul') {
        response.end(JSON.stringify(number1 * number2));
    } else if (operation == 'div') {
        response.end(JSON.stringify(number1 / number2));
    } else {
        response.end("Invalid Operation");
    }
});
```

Query Strings (The Standard Approach)

Instead of writing complex manual extraction functions, the standard approach relies on query strings and prebuilt libraries.

- **Standard Format:** Standard APIs append data to the URL using a query string format, such as ?ids=bitcoin&vs_currencies=inr or ?num1=10&num2=20.
- **Automatic Parsing:** Node.js includes a built-in url module that automatically converts this string into a structured JavaScript object.
- **url.parse(request.url, true):** Generates an object containing a pathname (e.g., "/add") and a query object ({ num1: 10, num2: 20 }), making data extraction significantly easier.

```
const http = require('http');
const url = require('url');

const server = http.createServer((request, response) => {

    const parsed = url.parse(request.url, true);

    const operation = parsed.pathname.slice(1);
    const number1 = Number(parsed.query.num1);
    const number2 = Number(parsed.query.num2);

    // Operation logic
});

server.listen(3000, () => {
    console.log("Server is listening at port 3000");
});
```

Modular Architecture

Large-scale backend systems, like Amazon, cannot be maintained in a single JavaScript file. A massive monolithic file containing hundreds of thousands of lines leads to poor readability, variable and function name conflicts, and accidental changes when multiple engineers work concurrently on features like authentication, payments, or emails.

To solve this, code is divided into separate modules. A **module** is simply code written in a separate file, which can be exported and utilized by other files.

CommonJS (CJS) – The Legacy Method

CommonJS is the older module standard used in Node.js.

- **Syntax:** Code is exported using module.exports and imported using require().
- **Export Behaviour:** By default, module.exports is an empty object. Multiple functions can be exported by attaching them to this object. However, assigning a new object to module.exports overwrites any previous exports. Exports should generally be placed at the end of the file. Destructuring can be used when importing the object.
- **Synchronous Loading:** Behind the scenes, require() loads code sequentially from secondary storage into RAM. Because JavaScript is single-threaded, CommonJS loading delays execution.

```
const {payment, hello} = require("./second.js");

console.log("Payment is processing...");

payment(500);
hello();
```

```
function payment(num) {
  console.log(`Payment of ${num} is completed`);
}

function hello() {
   console.log("Hello");
}

module.exports = {payment, hello};
```

ECMAScript Modules (ESM) – The Modern Standard

ES Modules represent the current industry standard, addressing the performance issues of CommonJS.

- **Syntax:** Functions are shared using export and import.
- **Asynchronous Loading:** ESM loads files in parallel, allowing asynchronous loading before execution begins, which significantly improves speed.
- **Enabling ESM:** Node.js defaults to CommonJS. ESM must be explicitly enabled either by changing file extensions to .mjs or by adding "type": "module" to the package.json file (which is the preferred practice).
- **Export Types:** ESM allows functions to be exported inline. It also features a default export, restricted to one per file, which can be imported directly by its name.

```
import hello, {add, sub} from "./second.mjs"

hello();
add(2, 2);
sub(2, 2);

console.log("I am complete!");
```

```
export default function hello() {
   console.log("Hello");
}
function add() {
   return a + b;
}
function sub() {
   return a - b;
}
export{add, sub};
```

require and import are not native to core JavaScript; they are features provided by the Node.js runtime or the browser.